

Project title:

Implementation of a Student Registration
System in a college

Designing and implementing a student registration
database system for educational institutions.

Chosen sector: Education Sector

Members:

Shong Qien Buo(M44100768), Irwin Tan(M44100798) and
Calvine Joash De Silva (M44100604)

Introduction

- Overview: The education sector plays a major role in developing knowledge and skills among individuals in society. It includes different types of institutions such as schools, colleges, universities, and training centers that provide learning opportunities for students at various levels. With the growth of technology and digital learning environments, the education sector has gradually adopted more computerized systems to support both teaching and administrative activities.
- Importance of database systems in this sector: Automation efficiency, Data integrity, security and regulatory compliance

Identified DBMS System

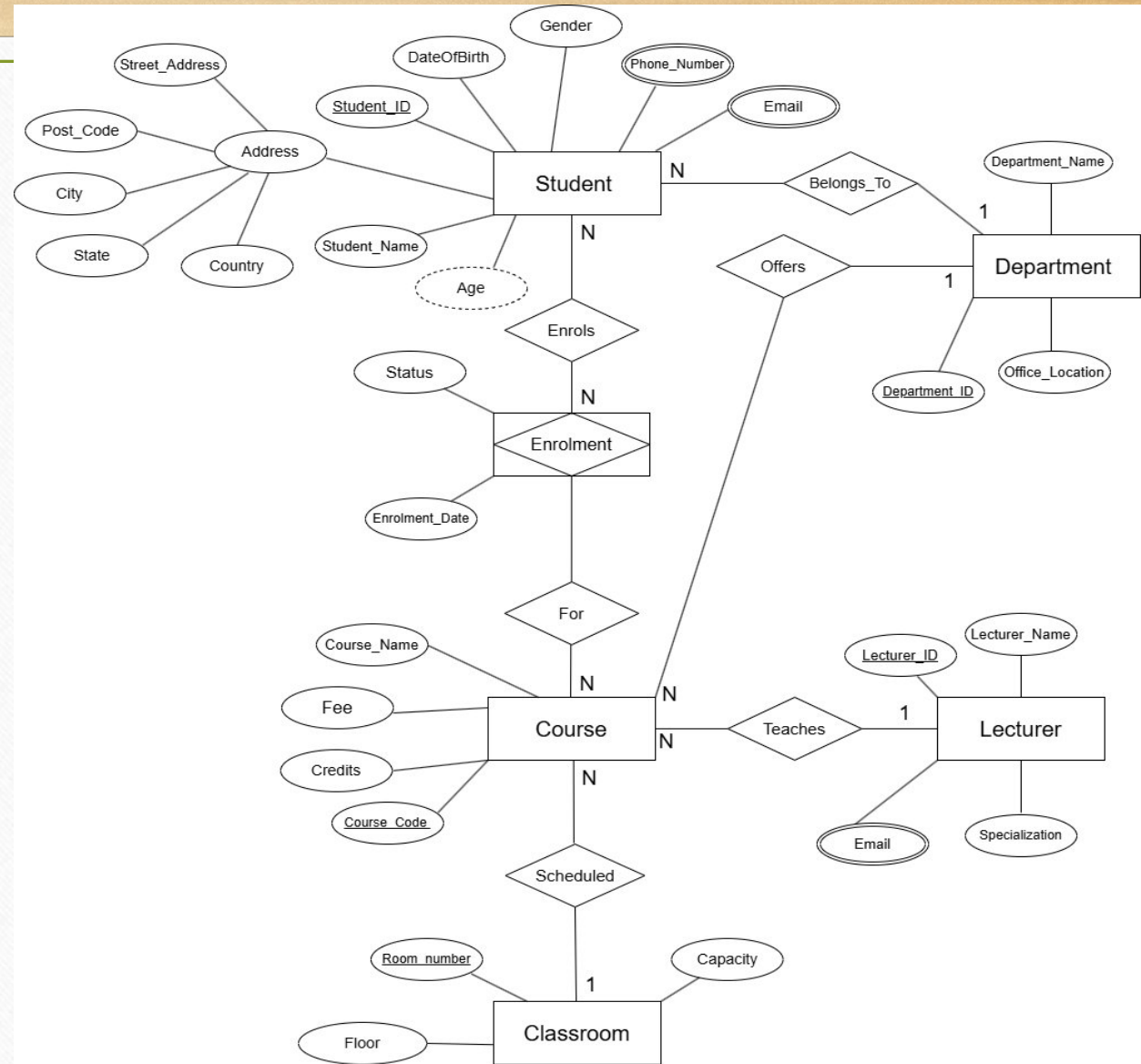
- Name of the system: Student Registration System
- Description: A Student Registration System is a digital platform used by educational institutions to manage student enrolment and academic records more efficiently. It helps educational institutions organize and manage student information in a more structured and reliable way.
Function: Manage student enrolment and academic record, can record student details and supports better coordination between different departments

Key Features & Components

- 1. Student Registration and Profile Management
- 2. Course and Curriculum Management
- 3. Online Self-Enrolment and Course Selection
- 4. Faculty and Classroom Scheduling (Timetable)
- Components: Student Registration System have 6 entities, which follows database components to operate smoothly in educational system. Student records such as name, age, date of birth, id and gender is recorded to manage the student profiles. StudentID is a unique identifier which allows the student to have a uniquely identifiable that is not the same from other students. For department, it contains the information's like departmentID, name and office location.

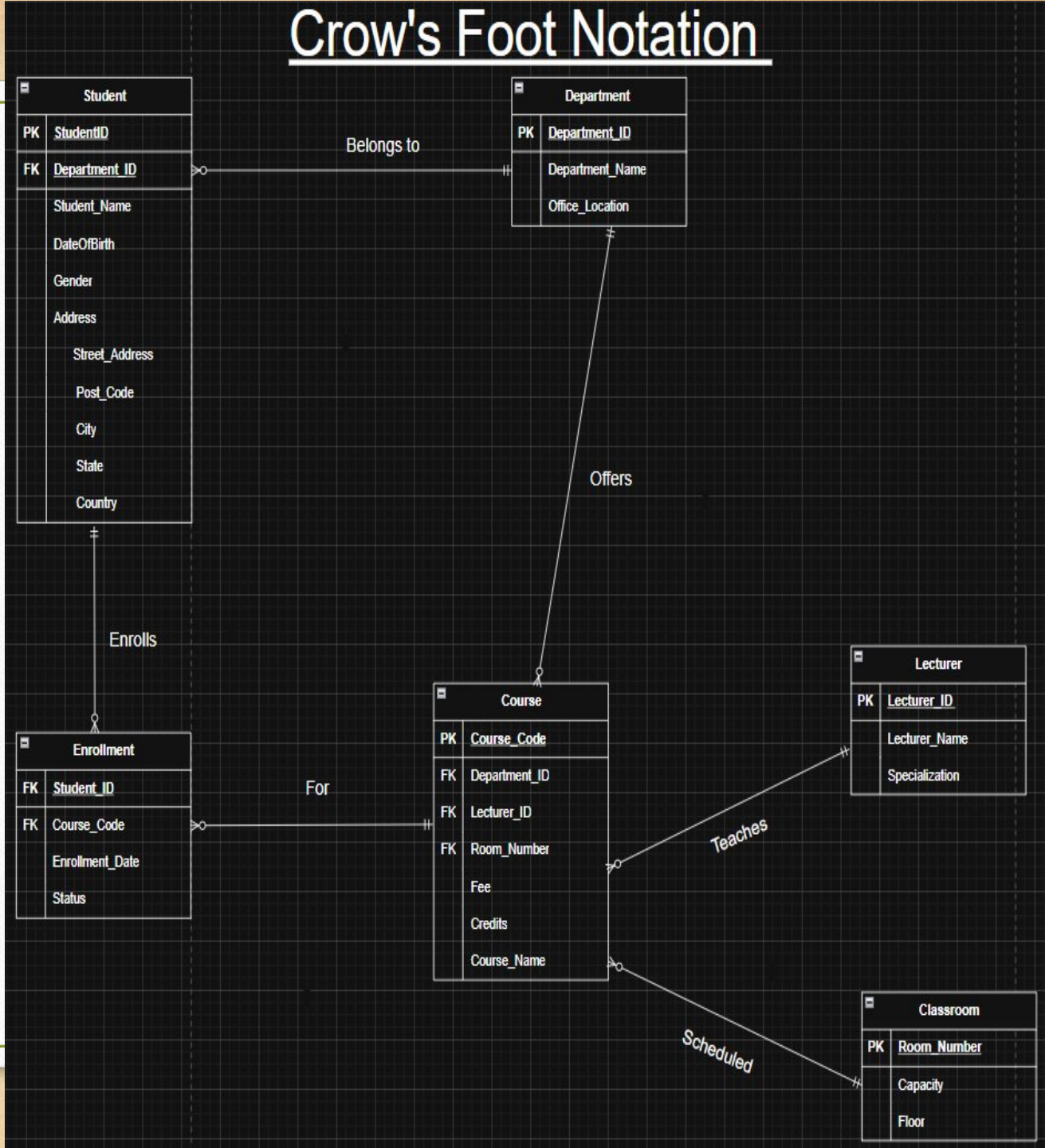
ERD (Entity-Relational Diagram)

1. Chen Notation



ERD (Entity- Relational Diagram)

2. Crow's Foot Notation



Database Creation

- How the database was created using DBMS (MYSQL) ☾
- Mention about table, primary keys, foreign keys

CRUD OPERATION

Step 1: Create (INSERT)

Example 1: Created Student Table

```
INSERT INTO Student (StudentID, Student_Name, DateOfBirth, Gender, Street_Address, Post_Code, City, State, Country,Department_ID)
VALUES
```

```
(1, 'Shong Qien Buo', '2007-08-10', 'Male', '56, Bandar Bukit Tinggi 2, Lorong Batu Nilam 33A', '41299', 'Klang', 'Selangor', 'Malaysia', 'DIPBUSS'),
(2, 'Irwin Tan', '2007-01-02', 'Male', '89, Bandar Bukit Bayu, Lorong Batu Nilam 22B', '32440', 'Subang', 'Terengganu', 'Malaysia', 'DIPCOMM'),
(3, 'Calvine Joash', '2007-05-29', 'Male', '34, Bandar Setia Alam, Lorong 44H', '45666', 'Puchong', 'Perak', 'Malaysia', 'DIPENGR'),
(4, 'Wong Yu Herng', '2007-08-11', 'Male', '66, Bandar Bukit Kepong, Lorong 67D', '56789', 'Kuala Lumpur', 'Perlis', 'Malaysia', 'DIPHOSP'),
(5, 'Qandeel Asif', '2006-11-06', 'Female', '99, Bandar Bukit Raja, Lorong Batu Nilam 29F', '41430', 'Cheras', 'Negeri Sembilan', 'Malaysia', 'DIPXDCS');
```

[illegible]

```

DROP TABLE IF EXISTS Student;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Student_Name VARCHAR(100) NOT NULL,
    DateOfBirth DATE NOT NULL,
    Age INT NULL,
    Gender VARCHAR(10),
    Street_Address VARCHAR(100),
    Post_Code VARCHAR(10),
    City VARCHAR(50),
    State VARCHAR(50),
    Country VARCHAR(50),
    Department_ID VARCHAR(100),
    FOREIGN KEY (Department ID) REFERENCES Department(Department ID) ON DELETE CASCADE
);

```


CRUD Operations

```
• DROP TABLE IF EXISTS ContactNumber;  
• CREATE TABLE ContactNumber(  
    StudentID INT NOT NULL,  
    Phone_Number VARCHAR(20) NOT NULL,  
    PRIMARY KEY (StudentID, Phone_Number),  
    FOREIGN KEY (StudentID)  
    REFERENCES Student(StudentID)  
    ON DELETE CASCADE  
);
```

```
INSERT INTO ContactNumber(StudentID,Phone_Numbe  
VALUES ( 001,'012376-5581'),  
        ( 002,'016662-5348'),  
        ( 002,'012661-6181'),  
        ( 003,'012377-0583'),  
        ( 004,'012966-3996'),  
        ( 005,'014678-9877');
```

Result Grid | Filter Rows:

	StudentID	Phone_Number
▶	1	012376-5581
	2	012661-6181
	2	016662-5348
	3	012377-0583
	4	012966-3996
	5	014678-9877

Step 1: Create (INSERT)

Example 2: Created Contact Number Table

CRUD Operations

Step 1: Create (INSERT)

Example 3: Created Email Address
Table

```
DROP TABLE IF EXISTS EmailAddress;  
CREATE TABLE EmailAddress(  
    StudentID INT NOT NULL,  
    Email VARCHAR(100),  
    PRIMARY KEY(StudentID,Email),  
    FOREIGN KEY(StudentID) REFERENCES Student(StudentID) ON DELETE CASCADE  
);
```

```
INSERT INTO EmailAddress(StudentID,Email)  
VALUES (001,'jaspershong2006@gmail.com'),  
       (001,'M44100768@student.uow.edu.my'),  
       (002,'irwin67@gmail.com'),  
       (003,'calvine88@gmail.com'),  
       (004,'wong23@hotmail.com'),  
       (005,'asif123@gmail.com');
```

Result Grid		Filter Rows:	Edit:
	StudentID	Email	
▶	1	jaspershong2006@gmail.com	
	1	M44100768@student.uow.edu.my	
	2	irwin67@gmail.com	
	3	calvine88@gmail.com	
	4	wong23@hotmail.com	
	5	asif123@gmail.com	
•	NULL	NULL	

CRUD Operations

Step 2: Read (SELECT)

Example 1: Filtered to find a specific StudentID and Student_Name from Student table

```
SELECT StudentID, Student_Name  
FROM Student  
WHERE StudentID = 1;
```

Result:

	StudentID	Student_Name
▶	1	Shong Qien Buo
•	NULL	NULL

CRUD Operations

Step 2: Read (SELECT)

Example 2: Filtered to find entire StudentID and email from Student Table based on ascending order

```
SELECT s.Student_Name, ea.Email  
FROM Student s  
JOIN EmailAddress ea ON s.StudentID = ea.StudentID  
ORDER BY Student_Name ASC;
```

Result:

	Student_Name	Email
►	Calvine Joash	calvine88@gmail.com
	Irwin Tan	irwin67@gmail.com
	Qandeel Asif	asif123@gmail.com
	Shong Qien Buo	jaspershong2006@gmail.com
	Shong Qien Buo	M44100768@student.uow.edu.my
	Wong Yu Herng	wong23@hotmail.com

CRUD Operations

Step 3: Update (UPDATE)

Example 1: Updating a Street_Address from a specific StudentID

UPDATE Student

```
SET Street_Address = '12, Jalan Flora, Lorong Batu Nilam 67',
    City = 'Shah Alam',
    Post_Code = '40000'
WHERE StudentID = 1;
```

[illegible]

CRUD Operations

Step 3: Update (UPDATE)

Example 2: Updating EmailAddress's table Email value for Student with condition if StudentID and Email

	StudentID	Email
▶	1	M44100768@student.uow.edu.my
	1	shong_new2026@gmail.com
	2	irwin67@gmail.com
	3	calvine88@gmail.com
	4	wong23@hotmail.com
	5	asif123@gmail.com
*	NULL	NULL

```
UPDATE EmailAddress
SET Email = 'shong_new2026@gmail.com'
WHERE StudentID = 1 AND Email = 'jaspershong2006@gmail.com';
```


CRUD Operations

Step 4: Delete (DELETE)

Example 1: Deleting his personal email but keeps his student for StudentID and Email

```
DELETE FROM EmailAddress
```

```
WHERE StudentID = 1 AND Email = 'jaspershong2006@gmail.com'
```

	StudentID	Email
▶	1	M44100768@student.uow.edu.my
	2	irwin67@gmail.com
	3	calvine88@gmail.com
	4	wong23@hotmail.com
	5	asif123@gmail.com
•	NULL	NULL

CRUD Operations

Step 4: Delete (DELETE)

Example 2: Deleting a specific Student's information from studentID which is 3.

```
DELETE FROM Student
WHERE StudentID = 3;
```

[illegible]

Advanced SQL Objects

```
DROP VIEW IF EXISTS StudentDepartmentView;  
CREATE VIEW StudentDepartmentView AS  
SELECT s.StudentID, s.Student_name,s.Gender, d.Department_name,d.Department_ID  
FROM Student s  
JOIN Department d  
ON s.Department_ID = d.Department_ID;
```

	StudentID	Student_name	Gender	Department_name	Department_ID
►	1	Shong Qien Buo	Male	School Of Business	DIPBUSS
	2	Irwin Tan	Male	School Of Communication	DIPCOMM
	3	Calvine Joash	Male	School of Engineering Technology	DIPENGR
	4	Wong Yu Heng	Male	School Of Hospitality	DIPHOSP
	5	Qandeel Asif	Female	School Of Computer Science	DIPXDCS

Example 1: Student and Department View (View)

Description: Display student details and Gender along with their corresponding department without repeatedly writing JOIN statements.

It can improve the readability and usability of the database

Advanced SQL Objects

- Example 2: Update Course Fee (Stored Procedure)

Description: Purpose of stored procedure is to modify multiple attributes in one execution.

```
DELIMITER //
```

```
CREATE PROCEDURE UpdateCourseDetails(  
    IN courseCode VARCHAR(10),  
    IN newRoom VARCHAR(10),  
    IN newFee DECIMAL(10,2)  
)
```

```
BEGIN  
    UPDATE Course  
    SET Room_Number = newRoom,  
        Fee = newFee  
    WHERE Course_Code = courseCode;  
END //
```

```
DELIMITER ;
```

```
CALL UpdateCourseDetails('COMM2014', 'R004', 1000.00);
```

	Course_Code	Course_Name	Credits	Fee	Department_ID	Room_Number	Lecturer_ID
▶	BUSS2013	Principles Of Management	4	1350.00	DIPBUSS	R001	L001
	COMM2014	Mass Communication Theory	2	1000.00	DIPCOMM	R004	L002
	ENGR1014	Introduction to Mechanical Engineering	3	1000.00	DIPENGR	R003	L003
	ENGR1023	Introduction to Hospitality	2	850.00	DIPENGR	R004	L004

Advanced SQL Objects

- Example 3: Validate Course Fee (Trigger)

- Description: Uses the trigger objects to prevent invalid or unrealistic course fees from inserted

```
DELIMITER //
```

```
CREATE TRIGGER ValidateCourseFee
```

```
BEFORE INSERT ON Course
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF NEW.fee < 100 THEN
```

```
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Course fee must be at least 100';
```

```
    END IF;
```

```
END //
```

```
INSERT INTO Course (Course_Code, Course_Name, Credits, Fee, Department_ID, Room_Number, Lecturer_ID)
```

```
VALUES ('COMM456', 'Introduction to Public Relation', 2, 20.00, 'DIPCOMM', 'R006', 'L006');
```

```
DELIMITER ;
```



54 15:47:06 INSERT INTO Course (Course_Code, Course_Name, Credits, Fee, Department_ID, Room_Number, Lecturer_ID)... Error Code: 1644. Course fee must be at least 100

Advanced SQL Object

- Example 4: Calculate Discounted Fee (Function)
- Description: Calculate the total fee after applying discount to a course fee.

```
SELECT
    Course_Code,
    Course_Name,
    CONCAT(20,'%') AS Discount_Percentage,
    Fee AS Original_Fee,
    CalculateDiscountedFee(fee, 20) AS Discounted_Price
FROM Course
WHERE Course_Code = 'COMM2014';
```

```
DELIMITER //
```

```
CREATE FUNCTION CalculateDiscountedFee(
    originalFee DECIMAL(10,2),
    discountPercent DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE discountedFee DECIMAL(10,2);

    SET discountedFee = originalFee - (originalFee * discountPercent / 100);

    RETURN discountedFee;
END //
```

```
DELIMITER ;
```

	Course_Code	Course_Name	Discount_Percentage	Original_Fee	Discounted_Price
►	COMM2014	Mass Communication Theory	20%	1000.00	800.00

Real-Life Company Examples

For the current education, Database Management Systems have been used in supporting numeric academic systems. Here are some educational institutions are being used

- University student management systems. (Provides students with opportunity to access their own information)
- Course registration systems (Used in supporting numerous academic or educational systems)

Limitations / Challenges

The current challenges that the Student Registration System are facing are

- Data inconsistency : Duplicate or contradicting student information can be the result of bad data management or the lack of constraints in the system
- Security concerns : Sensitive information like students' personal information, and financial information is prone to cyber attacks. Without proper authentication mechanisms, unauthorized users can gain access, leading to data breaches and loss of trust
- Maintenance : During peak periods such as course registration, the system can experience slow performance, delays, and even failure

Suggestions for Improvement

The Student Registration System can be improved by implementing the changes such as:

- Implementing cloud-based database : The adoption of cloud platforms enables the system to scale up the resources during such times of high traffic. This ensures the smooth functioning of the system even when there is a large number of students accessing the system at the same time
- Using Data Analytics : Data analytics can be used for analysing the performance of students and finding patterns in the results AI features can provide recommendations for courses and can be used for early alerts on poor performance by students

REFERENCES

- Mukul, E., & Büyüközkan, G. (2023). *Digital transformation in education: A systematic review of education 4.0. Technological Forecasting and Social Change*, 194, 122664. <https://doi.org/10.1016/j.techfore.2023.122664>
- Anika, A., & Kumar, D. (2023). *Streamlining Student Enrollment: University's College Registration System*. Rawat Prakashan. A related chapter similarly describing manual processes as "prone to mistakes" <https://lrcdrs.bennett.edu.in/items/6fc43717-5acf-43e4-8e21-ba58d181b9d0>
- Kaka, A. R. (2023, June 17). *Unleashing the potential of cloud databases in education; enhancing collaboration, accessibility*. The Financial Express. <https://www.financialexpress.com/jobs-career/education-unleashing-the-potential-of-cloud-databases-in-education-enhancing-collaboration-accessibility-3129298/>
- Almeshmes, I. A. H. (2014). *Design and implement of database student information management system in College of Medicine – University of Diyala* (Unpublished master's thesis). Çankaya University. <http://hdl.handle.net/20.500.12416/324>
- The Design and Implementation of Student Academic Record Management System. (2011). *Airiti Library*. <https://www.airitilibrary.com/Article/Detail?DocID=20407467-201108-201411110030-201411110030-707-712>
- **Building Course Registration Project with Azure SQL Database.** (2024). *Azure SQL Dev Corner* (Microsoft). <https://devblogs.microsoft.com/azure-sql/course-registration-project-with-azure-sql-database/>

THANK YOU!!

HAVE A NICE DAY !!